

Restoring Degraded Inspection Images

Joint speckle/Gaussian denoising and 2x super-resolution, built on a degradation model recovered from the data itself.

KLA Problem Statement | SEMICON India Hackathon 2026

Team submission · github.com/23f2001033/kla-restore

The restoration task

Three degradations, applied in an undisclosed order

What we are given

- 3200 paired samples: clean GT and its degraded counterpart
- GT 256×256 float32 in [0,1]; NoisyLR 128×128
- Degradations: speckle noise, additive Gaussian noise, downsampling
- Order undisclosed — the model must not depend on knowing it
- NoisyLR deliberately exceeds [0,1] (observed [-0.28, 2.16])

What is scored

- Restoration quality: PSNR, SSIM and LPIPS on hidden ground truth
- End-to-end throughput on an NVIDIA H100 — including script startup, disk I/O and pre/post-processing
- Training and compute hygiene: reproducibility, clean experiments
- Hidden test set spans in-distribution and out-of-distribution content

23.18 dB

bicubic upsample of NoisyLR
the do-nothing baseline

32.25 dB

perfect denoise + bicubic
interpolation-only ceiling

> 32.25 dB

target for a model that truly learns SR
must beat interpolation, not match it

We recovered the degradation model from the data

Rather than guessing at it — every number below is reproducible

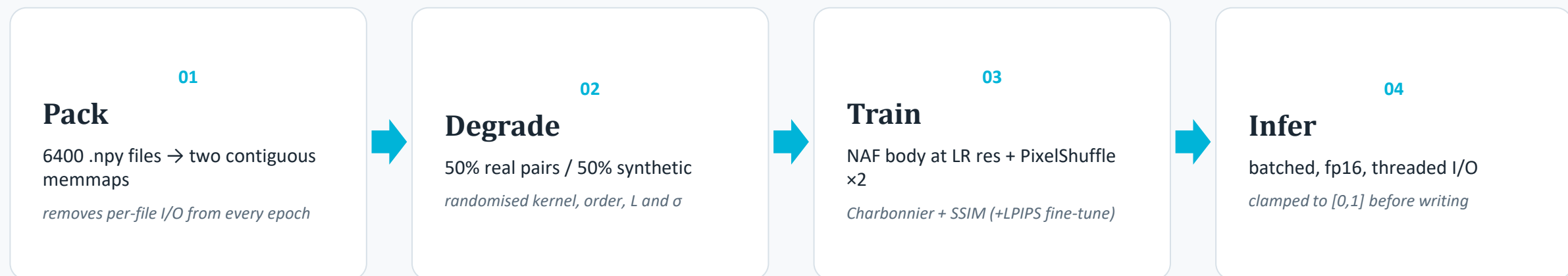
Property	Finding	Evidence
Downsampling	2×2 area (box) average	residual std 0.0910 vs 0.0924 bicubic, 0.1006 nearest
Noise placement	applied after downsampling	residual is spatially white (lag-1 autocorr -0.05)
Speckle family	multiplicative Gamma	skew of r/d = +0.357 vs Gamma's predicted $2/\sqrt{36} = +0.333$
Speckle strength	L: p5 22, median 36, p95 57	per-image fit of $\text{var}(r) = d^2/L + \sigma^2$
Gaussian strength	σ : median 0.021, p95 0.073	21.3% of images below 0.005

Why this mattered

Multiplicative Gaussian speckle would be symmetric; the measured +0.357 skew is what pins the family to Gamma. Fixing the forward model is what let us generate synthetic pairs that match the real distribution instead of a plausible-looking guess — the first attempt was 50% too noisy (residual std 0.137 vs 0.091) and would have biased the model toward over-smoothing.

End-to-end pipeline

One model, one forward pass, no staging



Scale-agnostic by construction

The network is fully convolutional with no fixed-size assumption, so the same weights handle 128→256 and 256→512. The brief states test data may contain either. Verified end-to-end: a 256×256 input emits 512×512 with no shape assertion.

`python inference.py --input_dir <dir> --output_dir <dir>`

Synthetic degradation engine

Sampled from the measured distribution, then deliberately widened

How parameters are sampled

(L, σ) are drawn from empirical quantile tables fitted on all 3200 real pairs, then multiplied by a random widening factor so the tails extend past what was observed. This reproduces the true distribution by construction rather than assuming a parametric form.

Kernel $\in \{\text{area, bilinear, bicubic, gaussian+stride}\}$; order randomised across the three degradations.

Source	residual std	skew r/d	per-image p10 / p50 / p90
real pairs	0.0852	+0.367	0.045 / 0.079 / 0.114
synthetic (exact)	0.0945	+0.299	0.050 / 0.081 / 0.135
synthetic (widened)	0.1010	+0.483	0.045 / 0.086 / 0.145

Synthetic pairs match real statistics closely; the widened variant extends both tails for OOD robustness.

Training mix and augmentation

- 50% real provided pairs (anchors the exact test distribution) / 50% synthetic (widens it)
- 64×64 LR crops → 128×128 GT, random flips and 90° rotations; crop-based training is what makes the model scale-agnostic
- Synthetic LR is never clipped — the real data is not either, and clipping would create a train/test mismatch on exactly those values

Architecture

NAFNet-style body at LR resolution, single PixelShuffle $\times 2$ head

input (B, 1, H, W)

conv3 $\times$ 3 $\rightarrow$ width 64

16 $\times$ NAFBlock

depthwise conv + SimpleGate

simplified channel attention

conv3 $\times$ 3 (long residual)

conv3 $\times$ 3 $\rightarrow$ PixelShuffle(2)

conv3 $\times$ 3 $\rightarrow$ 1 channel

+ bilinear(input, $\times 2$)

output (B, 1, 2H, 2W)

Design rationale

- Body runs at LR resolution and upsamples once at the end — $\sim 4\times$ cheaper than a U-Net at output resolution, and throughput is scored
- NAF blocks reach near-transformer quality with pure convolution; SwinIR/Restormer gain little at 128 $\times$ 128 tiles and cost real H100 time
- No BatchNorm anywhere — channel-wise LayerNorm only, so the evaluator's batch size cannot change our outputs
- Per-block residual scales initialise at zero: a 16-block stack starts as identity and trains stably from scratch
- No GAN — the brief asks for restoration without hallucinating structure

0.68 M

parameters

4.8e-07

max |batch8 – batch1|
batch-size invariance verified

128 $\rightarrow$ 256 and 256 $\rightarrow$ 512

same weights, no reconfiguration

Loss and training setup

Pixel fidelity, structure and perception — weighted for what is scored

Objective

$L = \text{Charbonnier} + 0.15 \cdot (1 - \text{SSIM})$

+ 0.05 · LPIPS for the final 10% of steps

- Charbonnier over L2: better PSNR/SSIM trade-off and robust to the heavy-tailed speckle outliers
- Single-scale SSIM, not MS-SSIM — MS-SSIM needs ≥161 px and the GT crops are 128×128; it would fail or silently degrade
- LPIPS is scored, but VGG every step costs ~30% throughput — a short fine-tune captures most of the benefit

Setting	Value
Optimiser	AdamW, betas (0.9, 0.999), wd 1e-4
Learning rate	5e-4, 2000-step warmup, cosine to 1e-6
Batch / precision	32, AMP, channels_last
Stability	grad-norm clip 1.0, non-finite loss guard
Checkpointing	validate every 5000 steps, keep best by PSNR

Stability incident and fix

An initial run diverged to a non-finite loss exactly as warmup ended and the learning rate peaked. Root cause: AdamW was configured with betas=(0.9, 0.9); the fast second-moment decay made the per-parameter step size volatile enough that one gradient spike at peak LR corrupted the weights irrecoverably. Fixed to (0.9, 0.999), lowered the peak LR to 5e-4, and added a guard that skips non-finite batches and aborts after 20 consecutive ones rather than silently consuming the budget.

Experiment tracking and validation design

Reproducibility as a first-class deliverable

The validation split is group-aware

KLA's own slides show sample 000000 → source 0001.png and sample 000500 → source 0186.png: samples are ordered by source image, roughly 2-3 per source.

- A random per-index split therefore leaks near-duplicate content into training
- A contiguous tail slice hands validation sources seen nowhere else
- We hold out 10 evenly-spaced contiguous blocks of 32 — contiguous keeps same-source samples together, spacing covers all content types
- Split frozen in configs/base.yaml; $\text{train} \cap \text{val} = \emptyset$ asserted at load

What every run records

- run id, git hash, seed, step, LR, loss, all three metrics, parameter count and the full config as JSON — appended to results/run_log.csv at every validation
- Seeds fixed for Python, NumPy and Torch
- Every hyperparameter lives in configs/base.yaml; nothing hard-coded
- The checkpoint embeds its own model config, so inference rebuilds the architecture without reading any config file
- Schedule length is calibrated from measured throughput, so cosine reaches its minimum exactly when the budget ends

Pre-flight gate

train.py --overfit 2 --steps 2000 must exceed 35 dB on two images before any long run starts, and exits non-zero on failure so it can gate a script. Crop alignment is verified independently: aligned residual 0.078 vs 0.128 for a 1-pixel shifted control.

Restoration quality

Held-out validation split, 320 images never seen in training

Method	PSNR (dB)	SSIM	LPIPS
bicubic upsample (baseline)	22.94	0.5460	0.4478
perfect denoise + bicubic (ceiling)	32.25	—	—
this model	27.54	0.7594	0.3922

+4.60 dB

over the bicubic baseline

OOD probe

external images degraded with our engine
proxy for the hidden OOD split

Metric conventions — stated because implementations differ

PSNR: data_range=1.0 on the clipped prediction, averaged per image. SSIM: 11×11 Gaussian window, $\sigma=1.5$, data_range=1.0 — equivalent to skimage's gaussian_weights=True; skimage's default 7×7 uniform window yields a different number. LPIPS: LPIPS-VGG on 3-channel-replicated grayscale, inputs mapped to [-1, 1].

Throughput

Scored end-to-end: startup, I/O and pre/post-processing all count

What we optimised

- Only torch, numpy and argparse imported at module level — no yaml, lpips, skimage or matplotlib, each of which costs startup time for no benefit
- Model config travels inside the checkpoint, so there is no config parse
- Files read and written on a thread pool (numpy I/O releases the GIL)
- Inputs grouped by resolution and run in batches under inference_mode, with channels_last, pinned memory, non-blocking transfers and fp16
- torch.compile off by default — 30-60 s of compile will not amortise over a small test set; available behind --compile
- ×8 self-ensemble available behind --tta: ~+0.2 dB for ~8× the runtime, off by default since throughput is scored

Measured

21.7 ms

per image, model forward

46 images / second

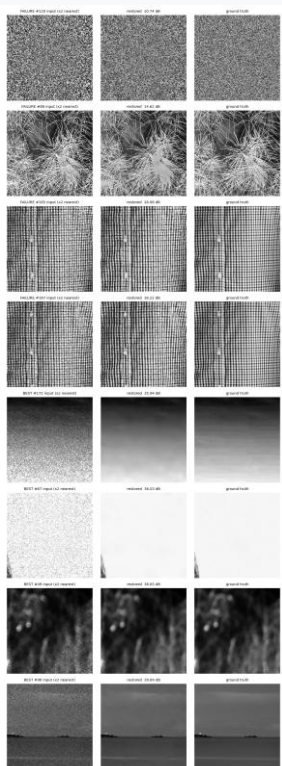
Full-pipeline wall clock is measured with the benchmark cell in the training notebook: it stages the validation split as loose .npy files and times inference.py from process start to the last file written — the same boundary KLA uses.

Hardware, batch size, driver, torch version and timing method are recorded alongside the number.

`time python inference.py --input_dir <dir> --output_dir <dir>`

Visual results and failure analysis

Best and worst cases from the held-out split



Dominant failure mode

Heavy speckle (low L) over fine high-frequency texture. Where the noise variance approaches the local signal variance, the model cannot separate texture from speckle and smooths both — the conservative choice, and the right one given the brief asks for restoration without hallucinating structure.

Limitations

- Trained on the released 256→128 pairs; the 512→256 case is handled by the fully-convolutional design and verified for shape correctness, but has no paired data to validate accuracy against
- The degradation engine assumes only the three named mechanisms
- No GAN or diffusion prior, so genuinely destroyed high-frequency detail is not invented — a deliberate accuracy/realism trade

Summary

Measured, not assumed

The degradation model was recovered from the data and validated statistically; synthetic pairs reproduce the real distribution.

Efficient by design

0.68 M parameters, body at LR resolution, single upsample — quality without paying H100 throughput for it.

Reproducible end to end

Config-driven, seeded, logged per validation; inference runs from a clean environment with two directory arguments.

External resources and licensing

No external datasets or pretrained weights are used for the restoration model itself — it is trained from scratch on the official KLA data plus synthetic pairs derived from that same data. lpips (BSD-2-Clause) supplies the VGG perceptual loss for the final fine-tune and for reporting; its VGG16 backbone carries ImageNet-pretrained weights shipped with the package. Training degrades gracefully to Charbonnier + SSIM if it is unavailable, and inference.py never imports it.

github.com/23f2001033/kla-restore

[README.md](#) · [train.py](#) · [inference.py](#) · [evaluate.py](#) · [analyze_degradation.py](#) · [configs/](#) · [src/](#) · [weights/](#) · [results/](#)